Wearch AI

Villaseñor Rivas Alejandro Rodrigo, Corona Salinas Alan Imanol, Zuñiga Lozano Racso Siseneg, Asesores: Dra. Ávila Cardenas Karla, Dr. Galvez Rodriguez Jorge De Jesus

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS, (CUCEI, UDG)

alejandro.villasenor4147@alumnos.udg.mx

alan.corona4138@alumnos.udg.mx

racso.zuniga7755@alumnos.udg.mx

karla.acardenas@academicos.udg.mx

jorge.galvez@academicos.udg.mx

Abstract— El acelerado crecimiento de la información científica dificulta la búsqueda, el análisis y la síntesis de conocimiento relevante, desafío que se ve agravado por la fragmentación de las herramientas empleadas en estos procesos. En respuesta, Wearch AI ofrece una solución integral que centraliza dichas tareas mediante un agente inteligente respaldado por herramientas especializadas. La plataforma integra tres componentes principales: (1) un módulo de descubrimiento de artículos científicos, (2) un sistema de visualización que muestra las relaciones entre trabajos mediante grafos interactivos, y (3) un chat para interactuar con el agente y profundizar en la información. Esta arquitectura modular permite optimizar el flujo de investigación, mejorar la comprensión contextual y acelerar la toma de decisiones basada en evidencia.

Palabras claves – Agente Inteligente, Asistente de Investigación, Retrieval-Augmented Generation (RAG), Embeddings, Procesamiento de Lenguaje Natural (PLN), Flujo de Trabajo Académico, MapReduce, LangChain, OpenAlex, Inteligencia Artificial.

Repositorio de código:
https://github.com/AlejandroVillasenor/ProyectoModular
Licencia legal código:
Apache License 2.0

I. Introducción

En la era digital actual, la investigación científica enfrenta un crecimiento en la producción de información sin precedentes. Cada día se publican miles de artículos, informes y documentos en repositorios académicos, revistas y plataformas de todo tipo, generando una sobrecarga de información que dificulta el proceso de búsqueda, análisis y síntesis de conocimiento. Aunque esta expansión representa un avance significativo para la ciencia, también plantea un reto: los investigadores deben dedicar una gran cantidad de tiempo a localizar, filtrar y organizar datos antes de poder enfocarse en el análisis crítico de la información realmente relevante.

Tradicionalmente, el proceso de revisión de literatura se ha desarrollado de manera manual. Esto implica consultar múltiples fuentes, descargar documentos, leer extensos textos y, posteriormente, clasificarlos de forma individual. Este método, además de demandar mucho tiempo, puede generar redundancias, omisiones y un uso ineficiente de los recursos disponibles. En consecuencia, los investigadores terminan invirtiendo esfuerzos en tareas mecánicas y repetitivas que podrían ser automatizadas mediante herramientas tecnológicas especializadas.

El avance de la inteligencia artificial y del procesamiento de lenguaje natural (PLN) ha abierto nuevas posibilidades para transformar este panorama. Actualmente, existen modelos capaces de comprender, resumir y relacionar información textual de manera semántica, lo que permite crear sistemas más inteligentes de búsqueda y análisis de datos. Sin embargo, la mayoría de las herramientas disponibles siguen siendo fragmentadas: algunas se enfocan en la gestión de referencias, otras en la búsqueda de artículos, y otras en la generación de resúmenes, sin integrarse en un flujo de trabajo unificado. Esta falta de cohesión tecnológica limita la eficiencia del proceso de investigación académica.

Ante esta problemática, surge Wearch AI, una plataforma de asistencia a la investigación que busca optimizar la revisión de literatura científica mediante la integración de inteligencia artificial, los agentes, la recuperación aumentada por generación (RAG) y el manejo de interfaces de programación de aplicaciones (APIs). La propuesta central de este proyecto consiste en ofrecer un entorno unificado donde el investigador pueda realizar búsquedas, analizar relaciones entre artículos, generar resúmenes y formular consultas contextuales en lenguaje natural, todo dentro de una misma plataforma. De este modo, se pretende reducir los pasos manuales, agilizar el acceso a la información y mejorar la comprensión global del campo de estudio.

El desarrollo de Wearch AI responde a la necesidad de contar con herramientas que no solo automaticen tareas, sino que además apoyen cognitivamente al investigador, permitiéndole centrarse en la interpretación y construcción de conocimiento. Este proyecto busca demostrar que la combinación de técnicas modernas de inteligencia artificial con fuentes bibliográficas abiertas puede transformar la forma en que se realiza la revisión de literatura, haciendo que el proceso sea más eficiente, coherente y accesible.

Por lo tanto, Wearch AI representa un paso hacia la modernización del flujo de trabajo académico, al integrar distintas fases del proceso investigativo dentro de una sola plataforma inteligente. Con este enfoque, se promueve un uso más estratégico del tiempo y se facilita el acceso a información relevante, contribuyendo al avance del conocimiento científico y al fortalecimiento de la práctica investigativa en la comunidad académica.

II. TRABAJOS RELACIONADOS

En el ecosistema de herramientas para asistencia a la investigación existe una amplia cantidad de aproximaciones que abordan desafíos específicos del flujo de trabajo académico: desde la búsqueda y descubrimiento de literatura hasta la visualización de relaciones entre artículos y la generación asistida de resúmenes. A continuación se examinan las plataformas y trabajos más influyentes que sirvieron como inspiración y base conceptual para Wearch AI, señalando las similitudes y diferencias con el objetivo de posicionar la contribución única y el valor diferencial que este proyecto aporta al campo.

- *NotebookLLM (Google):* propone una experiencia conversacional sobre documentos: el usuario carga textos y puede realizar preguntas, recibiendo respuestas contextualizadas y referenciadas en el documento. Su fortaleza radica en la integración fluida de múltiples documentos y en la capacidad conversacional. Wearch AI comparte esta motivación —facilitar consultas contextuales sobre recursos— pero su diferenciador es la arquitectura modular diseñada para integrarse con conectores bibliográficos externos (por ejemplo, OpenAlex) y la orquestación explícita de herramientas (scraping, RAG) mediante un agente maestro. Además, el enfoque de Wearch AI prioriza la visualización de relaciones bibliográficas como grafo, elementos pensados para investigación académica sistemática.
- *Connected Papers y ResearchGraph:* permiten explorar la literatura a través de grafos que muestran relaciones de similitud o citación entre artículos. Estas plataformas son muy eficaces para identificar antecedentes y trabajos relacionados rápidamente. Wearch AI incorpora esta idea de visualización, sin embargo, nuestra solución permite cruzar esta búsqueda y conexión de artículos (desde OpenAlex) con un agente, ofreciendo una experiencia enriquecida con inteligencia artificial.
- *OpenAlex*: es una fuente pública de metadatos bibliográficos y se utilizó como conector principal para obtener artículos similares, metadatos y relaciones de citación. La elección de OpenAlex permite a Wearch AI operar con datos abiertos y escalables. Varios proyectos y servicios adoptan OpenAlex por su amplitud y por ofrecer una API accesible; la contribución de Wearch AI consiste en integrar estas consultas bibliográficas directamente en el flujo de trabajo y en representar los resultados tanto textualmente como mediante un grafo interactivo que se sincroniza con las consultas del usuario.
- *ResearchRabbit y scite.ai:* Por un lado ResearchRabbit ofrece descubrimiento visual y seguimiento de literatura con orientación en comunidades de investigación; mientras que scite.ai aporta evidencia de citaciones y contexto citacional (si la cita apoya, contradice, o simplemente cita). Ambos servicios aportan valor en la evaluación de la relevancia y la confianza de una referencia. Wearch AI, en su versión actual, establece conexiones citacionales entre publicaciones, aunque estas se limitan a la relación bibliográfica y no al contenido de los artículos.

Wearch AI recopila las ideas más destacadas disponibles en el ecosistema de software académico para desarrollar una integración modular que facilite la evolución de soluciones prácticas para la investigación asistida por Inteligencia Artificial.

III. DESCRIPCIÓN DEL DESARROLLO DEL PROYECTO MODULAR

El presente apartado detalla el proceso de desarrollo y despliegue del asistente de investigación Wearch AI, desarrollado como un proyecto modular e integrable.

A. Metodología de trabajo

El desarrollo del proyecto se realizó bajo un enfoque iterativo e incremental, siguiendo principios inspirados en la metodología ágil, lo que permitió dividir el trabajo en etapas y realizar pruebas parciales tras cada módulo implementado.

El proceso comenzó con el desarrollo de las herramientas (tools), iniciando por aquellas dedicadas al scraping de contenido web, la generación de resúmenes mediante el enfoque MapReduce y la creación de embeddings para el sistema RAG basado en una base de datos vectorial. Posteriormente, se realizaron las herramientas que tienen interacción con la OpenAlex: buscar papers similares y

obtener información sobre un paper. Una vez definidas las herramientas, se construyó el agente con Langchain. Posteriormente, se desarrolló el backend con Flask, encargado de orquestar la interacción entre las herramientas y la interfaz del usuario. Con ello, se desarrolló la interfaz web, centrada en ofrecer una experiencia visual clara e intuitiva para el usuario. Finalmente, se realizó la integración completa del sistema, uniendo el agente, el backend y el frontend, lo que permitió obtener una plataforma funcional e interactiva.

Para la organización del trabajo y el seguimiento del progreso, se utilizó un tablero Kanban en GitHub Projects, donde las tareas se clasificaron en columnas de pendientes, en progreso y finalizadas. Esta herramienta permitió mantener una gestión eficiente del flujo de trabajo y una clara visibilidad del avance del equipo.

Además, se llevaron a cabo reuniones semanales en Discord con el propósito de revisar los avances, identificar problemas técnicos y planificar las actividades de la siguiente iteración. Este esquema de comunicación constante favoreció la colaboración, la toma de decisiones en conjunto y la integración progresiva de los diferentes componentes del sistema.

B. Requerimientos principales

El objetivo general del proyecto fue diseñar y desarrollar una plataforma modular asistida por inteligencia artificial orientada a apoyar a los investigadores en la exploración, análisis y síntesis de información académica, reduciendo el tiempo dedicado a la revisión del estado del arte y las fuentes bibliográficas. utilizadas como sustento teórico en sus investigaciones.

El sistema se planteó bajo el enfoque de un agente inteligente (Agentic AI) capaz de gestionar distintas herramientas especializadas (tools) para asistir al usuario durante todo el proceso de revisión de literatura. La idea central consistió en que el usuario pudiera subir sus recursos (PDF o URLs), generar un resúmenes automáticos, visualizar papers similares mediante la API de OpenAlex, y realizar consultas personalizadas en el chat, donde un agent master interpretaría la petición y seleccionaría la herramienta adecuada para responder.

1) Análisis SMART del objetivo general

Con el propósito de guiar el desarrollo del proyecto hacia metas claras y alcanzables, se empleó la metodología SMART, la cual facilita la definición de objetivos específicos, medibles, realistas, relevantes y delimitados en el tiempo. Este análisis facilitó la planificación del trabajo, la asignación de tareas y la evaluación del cumplimiento de los resultados esperados en cada fase del proyecto.

- Específico: se centra en la creación de una plataforma que combine inteligencia artificial y sistemas de recuperación de información para optimizar la búsqueda y análisis de recursos científicos.
- Medible: el grado de éxito se evalúa a través de la reducción del tiempo de búsqueda del estado del arte y el funcionamiento adecuado de las herramientas principales (agente, APIs, scraping, resumen y RAG).
- Alcanzable: técnicamente viable gracias al uso de Python-Flask para el backend, LangChain para la gestión del agente, y la API de OpenAlex para la búsqueda de papers similares.
- Relevante: contribuye directamente a la mejora de la eficiencia en la investigación científica y a la centralización del proceso de análisis documental en una sola plataforma.
- Temporal: se proyectó un periodo de 6 a 8 meses para alcanzar un prototipo funcional capaz de integrar las herramientas principales y demostrar su utilidad práctica.

2) Requerimientos principales del sistema

Para cumplir con el objetivo, se establecieron los siguientes requerimientos funcionales:

- Permitir la carga de documentos PDF y enlaces web como fuentes de información.
- Realizar la extracción automática del contenido de los recursos proporcionados..
- Generar un resumen automático del primer recurso subido por el usuario mediante un enfoque MapReduce.
- Implementar una base de datos vectorial para almacenar los embeddings del contenido de los recursos.
- Integrar un sistema RAG (Retrieval-Augmented Generation) que responda preguntas con base a los embeddings de los recursos subidos.
- Crear tools con la API de OpenAlex para obtener papers similares, o bien, obtener la información de un paper en específico.
- Incorporar un grafo interactivo que visualice las relaciones entre los papers similares junto a un sistema de búsqueda.
- Implementar un Agente Maestro (Agent Master) para orquestación de herramientas.

Los requerimientos no funcionales definidos fueron:

- Interfaz responsive, que se adapte correctamente a distintos dispositivos y tamaños de pantalla.
- La interfaz de usuario debe ser intuitiva y accesible, ofreciendo retroalimentación visual inmediata ante

cada acción, mediante el uso de alerts, indicadores de carga (loaders) y mensajes de confirmación.
- El sistema debe ser compatible con archivos de hasta 60 MB, garantizando la correcta gestión y procesamiento de documentos extensos.
- En caso de error o desconexión del servidor, el sistema debe mostrar mensajes informativos y permitir reintentos automáticos sin pérdida de datos.
- La arquitectura del sistema debe mantener una escalabilidad modular, permitiendo la integración de nuevas herramientas o mejoras sin necesidad de modificar la estructura base del proyecto.
- El sistema debe implementar un mecanismo de comunicación en tiempo real bidireccional que garantice la sincronización inmediata de estados entre el frontend y backend, permitiendo actualizaciones automáticas, notificaciones de eventos y manejo robusto de conexiones interrumpidas.

C. Tecnologías utilizadas

Para el desarrollo del proyecto se implementó una arquitectura cliente-servidor, en la cual el backend gestiona la lógica de procesamiento, las peticiones del usuario y la comunicación con los modelos de inteligencia artificial, mientras que el frontend proporciona una interfaz visual interactiva y adaptable para la interacción del usuario con el sistema.

- Frontend: desarrollado en HTML5, JavaScript y Tailwind CSS, con un diseño responsivo e intuitivo.
- Backend: implementado con Python-Flask, encargado de la gestión de peticiones, procesamiento de documentos y comunicación con las herramientas de IA.
- Agente: su arquitectura se diseñó en LangChain sobre el modelo GPT-4-mini, integrando diversas herramientas especializadas. La herramienta de RAG emplea el modelo Mistral y la base de datos vectorial Chroma; el módulo de resúmenes combina LangChain, scikit-learn (sklearn) y Mistral; mientras que la búsqueda de artículos científicos se realiza a través de la API de OpenAlex.
- Generación de embeddings: se realiza mediante el modelo bge-m3.
- Scraping y análisis de contenido: se emplearon Scrapy, Playwright y Trafilatura para la extracción de texto desde sitios web tanto estáticos como dinámicos.
- Visualización de papers similares: consumo de la API pública OpenAlex para obtener información de publicaciones relacionadas junto a D3.js para su representación gráfica.

El código fuente completo se encuentra disponible en el repositorio público del proyecto: https://github.com/AlejandroVillasenor/ProyectoModular

D. Pruebas realizadas

Durante el desarrollo se llevaron a cabo diferentes pruebas orientadas a validar la estabilidad y desempeño del sistema:

- Pruebas unitarias: se evaluaron funciones clave como la carga de recursos (PDF y URLs), extracción de texto y generación de resúmenes automáticos, asegurando que cada componente operará correctamente de manera independiente.
- Pruebas de carga y desempeño: se procesaron documentos de distintos tamaños para medir la capacidad de manejo de información en paralelo y tiempos de respuesta aceptables.
- Pruebas de integración: se verificó la correcta interacción entre las herramientas del agente, así como la comunicación entre backend y frontend.
- Evaluación del desempeño del agente: Se analizó la capacidad de respuesta y razonamiento del agente, así como evaluar las prompts realizadas para su funcionamiento.
- Pruebas de usuario y usabilidad: se evaluó la facilidad de uso de la interfaz, la claridad de la información presentada por el agente, la relevancia de los resúmenes, las respuestas informadas del RAG, y la efectividad de la búsqueda de papers similares, así como la extracción de su metada.
- Pruebas de usabilidad del motor de búsqueda integrado con OpenAlex y la conexión de papers similares visualizada con D3.js.

Los resultados mostraron que la plataforma funciona correctamente dentro de los límites establecidos, manteniendo una interacción fluida entre backend y frontend. La calidad del agente fue buena tras razonar y hacer uso adecuado de sus herramientas de generación de resúmenes, la búsqueda de papers similares y la interacción con el sistema RAG. Sin embargo, se identificaron áreas de mejora, como la indicación del recurso de origen de cada respuesta; pero en general, la plataforma demostró estabilidad, confiabilidad y capacidad de cumplir con los objetivos planteados en su versión mínima viable (MVP, por sus siglas en inglés).

E. Proceso de implementación

El proceso de implementación de Wearch AI se llevó a cabo bajo un enfoque de Arquitectura Modular Desacoplada, en el cual cada componente funcional fue desarrollado y verificado de forma independiente antes de su integración final en un entorno unificado.

La base de la implementación consistió en la definición estricta de módulos con responsabilidades bien delimitadas, lo que permitió un desarrollo concurrente, una mayor mantenibilidad y una integración flexible de componentes. Gracias a esta estructura, fue posible incorporar nuevas funcionalidades o sustituir componentes individuales (como modelos o frameworks) sin comprometer la estabilidad ni la coherencia del sistema. El diseño general se resume en la Tabla I.

TABLA I
DISEÑO MODULAR

Módulo	**Función principal**	**Beneficio de la modularidad**
Frontend (Interfaz de Usuario)	Gestiona la presentación de la información y la interacción con el usuario mediante una interfaz web dinámica y responsiva.	Permite desarrollar y modificar la interfaz sin afectar la lógica del servidor, garantizando una experiencia de usuario fluida y adaptable.
Backend (Servidor de Aplicación)	Gestión de endpoints, enrutamiento de peticiones y comunicación con los módulos del Core.	Funciona como un gateway ligero que facilita la integración y el despliegue de servicios.
Core (Agente de IA)	Procesamiento del lenguaje natural, orquestación de herramientas y generación de respuestas contextuales.	Posibilita la actualización del modelo de lenguaje (LLM) o del framework de agentes sin alterar el resto de componentes.
Core (Tools)	Agrupa las herramientas (tools) que dotan al sistema de capacidades extendidas, permitiendo la ejecución de tareas especializadas y la integración de funcionalidades complementarias al agente principal.	Facilita la incorporación, mejora o sustitución de herramientas sin afectar la estabilidad del sistema, asegurando flexibilidad, escalabilidad y evolución continua del proyecto.

Una vez definidos los módulos, el desarrollo del proyecto se estructuró en las siguientes etapas:

1) Diseño de la arquitectura y maquetado

En esta primera etapa se realizó la planificación conceptual y el diseño general de la arquitectura del sistema. Se identificaron las funcionalidades principales del agente y la interacción esperada entre los módulos que integran a la plataforma, priorizando la independencia y la comunicación mediante interfaces bien definidas. Este diseño sirvió como base para el desarrollo estructurado y el aseguramiento de la coherencia funcional del sistema. El sistema contempla las siguientes categorías e interfaces:

1. Interfaz principal (Chat y gestión de fuentes): Presenta un entorno conversacional donde el usuario interactúa con el agente capaz de asistirlo durante su investigación. Desde esta pantalla, es posible cargar documentos externos —como archivos PDF o enlaces web— que se integran de inmediato al contexto de análisis. Además, el sistema recomienda artículos similares tomando como referencia el tema central tratado en el chat, lo que favorece el descubrimiento de nuevo conocimiento relacionado.

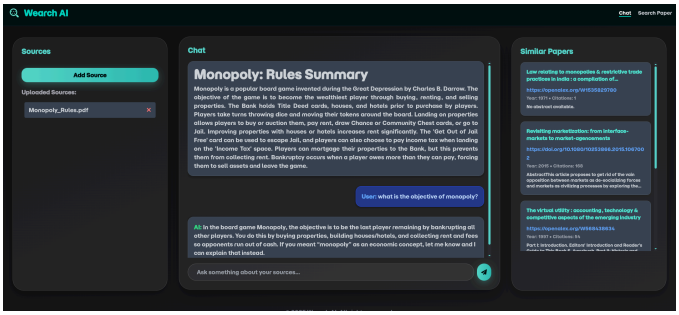

Fig. 1 Interfaz principal: Chat

2. Motor de búsqueda de artículos: Proporciona herramientas de consulta para localizar literatura científica de forma precisa. El usuario puede ingresar un DOI o realizar búsquedas por coincidencias en títulos, apoyándose en la base de datos de OpenAlex. Esta funcionalidad agiliza el acceso a artículos relevantes, ampliando el alcance de la investigación sin necesidad de abandonar la plataforma.

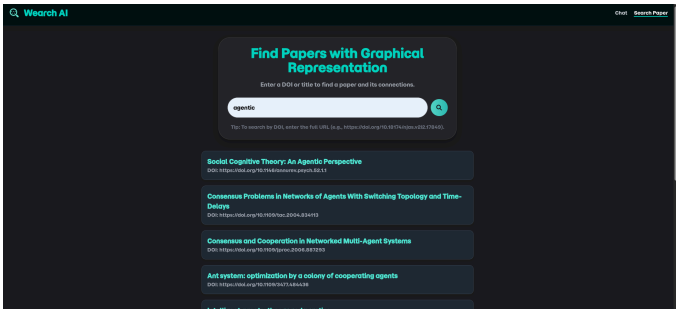

Fig. 2 Interfaz de búsqueda.

3. Interfaz de detalles y visualización relacional: Una vez seleccionado un artículo, el sistema despliega información detallada sobre su contenido, incluyendo metadatos esenciales. Adicionalmente, el usuario puede explorar su contexto bibliográfico mediante una representación gráfica interactiva que ilustra conexiones por cocitación con otros trabajos. Dichas relaciones se muestran como un grafo generado con

D3.js, donde cada nodo simboliza un artículo y puede seleccionarse para consultar su información. Esta visualización facilita comprender la relevancia del artículo dentro de la red científica y descubrir líneas de investigación relacionadas.

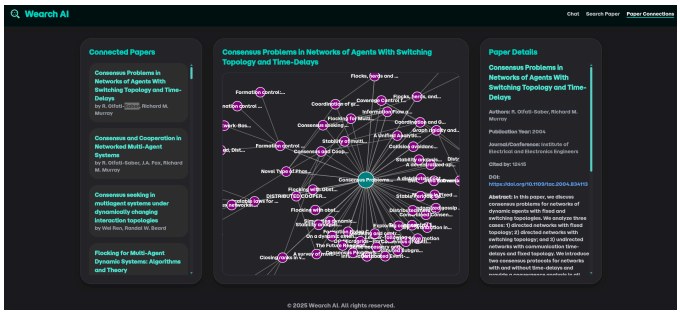

Fig. 3 Interfaz de conexión de artículos similares.

Con ello, se definieron las siguientes funcionalidades y módulos clave para el desarrollo del agente, tal como se ilustra en el diagrama:

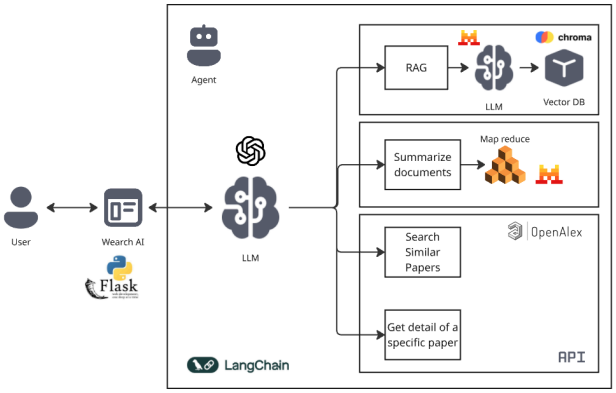

Fig. 4 Arquitectura del agente.

- RAG (Retrieval-Augmented Generation) con Chroma DB: Implementación de un flujo de recuperación y generación aumentada. El módulo RAG interactúa directamente con Chroma (Vector DB) para la gestión y búsqueda eficiente de embeddings de los documentos, permitiendo que el agente acceda a conocimiento específico y actualizado.
- Generación de Resúmenes: módulo encargado de procesar y condensar el contenido de documentos extensos o sitios web, aprovechando las capacidades del LLM (Mistral) para producir resúmenes coherentes y concisos. Emplea la técnica MapReduce, que divide los textos en fragmentos (chunks) generados mediante RecursiveCharacterTextSplitter y los procesa en paralelo mediante múltiples llamadas al modelo. Para textos de gran extensión, se incorpora la biblioteca scikit-learn (sklearn) con el algoritmo K-Means, lo que permite controlar el número de clusters o centroides y seleccionar los fragmentos más representativos del recurso antes de la síntesis final, mejorando así la precisión y eficiencia del proceso.
- Acceso a la API de OpenAlex: Se establecieron dos funcionalidades clave mediante la integración con la API de OpenAlex, enfocadas en el manejo de literatura académica:
    - Búsqueda de Artículos Similares (Search Similar Papers): Permite al agente consultar la base de datos de OpenAlex para identificar y recuperar documentos relacionados con un tema o artículo dado.
    - Obtención de Detalles de Artículos (Get detail of a specific paper): Facilita la recuperación de metadatos y detalles específicos de un artículo académico particular.

2) Inicialización del entorno de desarrollo y manejo de comunicación modular.

Se configuró el entorno de trabajo en Python utilizando Flask como servidor principal, aprovechando su ligereza, flexibilidad y compatibilidad con aplicaciones web modernas. En esta etapa se estableció la estructura base de las API-REST, definiendo los mecanismos generales para la comunicación entre módulos internos y servicios externos. Este trabajo inicial proporcionó los cimientos necesarios para garantizar un flujo de datos consistente y un desarrollo posterior ordenado.

3) Creación de las APIs internas para la integración con los servicios de terceros

Posteriormente, se diseñaron y desarrollaron APIs internas especializadas para gestionar la interacción directa con plataformas externas. Estas interfaces permiten la consulta, extracción y procesamiento de información académica, convirtiéndose en uno de los principales canales de adquisición de conocimiento del sistema. Su arquitectura modular favorece la reutilización, facilita su integración en los distintos componentes de la plataforma y habilita una comunicación eficiente con la interfaz de usuario.

4) *Diseño del RAG*

Se implementó una estrategia de búsqueda aumentada por recuperación (RAG, por sus siglas en inglés), cuyo propósito es permitir al modelo generar respuestas fundamentadas en información relevante extraída directamente de las fuentes proporcionadas por el usuario.

Durante el proceso de consulta, la pregunta del usuario se transforma en un vector y se compara con los embeddings almacenados en la base de datos para identificar los fragmentos con mayor similitud semántica. Los resultados más relevantes se utilizan como contexto para el modelo de

lenguaje, que genera una respuesta coherente, fundamentada y directamente relacionada con la información presente en los recursos del usuario.

5) Implementación de las herramientas para nuestro agente

En esta etapa se implementaron las herramientas del agente siguiendo la estructura definida por nuestro framework base. Gracias a ello, el agente cuenta con un conjunto de capacidades bien delimitadas, así como con lineamientos específicos sobre cómo debe emplear cada herramienta según los distintos casos de uso. Este proceso consolidó una arquitectura basada en agentes, flexible, modular y orientada a la resolución de tareas complejas de investigación, optimizando tanto su escalabilidad como su capacidad de adaptación.

6) *Prompt Engineering: la filosofía del agente.*

Esta fase fue una de las más importantes del desarrollo, ya que definió el comportamiento y la filosofía central del agente mediante técnicas de *prompt engineering*. Para estructurar su razonamiento y mantenerlo alineado con los objetivos del proyecto, se implementó un Routing Decision Tree, asegurando que tanto sus decisiones como su interacción con el usuario sigan rutas lógicas y coherentes con el diseño planteado.

Además, se incorporaron post-validaciones de razonamiento mediante el patrón Singleton, lo que garantiza que el agente reflexione e itere sobre sus propios procesos de forma controlada y eficiente.

La filosofía definida para este agente se fundamenta en el principio de que, dentro de la investigación académica, la confianza es el eje rector. Por ello, el agente prioriza la honestidad y la transparencia: reconoce cuando no cuenta con acceso o certeza sobre la información antes que ofrecer respuestas imprecisas. Así, su propósito no es únicamente generar contenido, sino actuar como un socio de investigación confiable, comprometido con la precisión, la responsabilidad y el rigor académico.

7) Desarrollo de las interfaces de usuario

En esta etapa nos enfocamos exclusivamente en el desarrollo de las interfaces definidas para las tres fases que conforman al agente. Se implementaron funciones comunes para gestionar los distintos elementos visuales, complementadas con manejadores de eventos basados en funciones asíncronas, con el objetivo de mantener un desempeño óptimo dentro de la plataforma.

Un aspecto particularmente relevante fue el procesamiento de los mensajes, especialmente aquellos generados por el propio agente. Algunos mensajes requerían formatos específicos o la capacidad de interactuar directamente con el código fuente de la página para detonar eventos y modificar la interfaz conforme a las solicitudes del usuario. Esta lógica permitió enriquecer la experiencia del usuario y asegurar una comunicación dinámica y contextual entre la interfaz y el agente.

8) Optimización de la comunicación entre módulos:

Finalmente, se implementó un sistema de Server-Sent Events (SSE) para mantener comunicación en tiempo real entre el servidor y el usuario. Esto permitió notificar desde el backend el progreso de tareas como la generación de resúmenes. Además, se estableció un mecanismo de almacenamiento temporal de recursos, mejorando la eficiencia y reduciendo los tiempos de respuesta del sistema.

Como resultado, Wearch AI se consolidó como una herramienta modular, extensible y adaptable, capaz de integrar diversas tareas de búsqueda, análisis y generación de conocimiento en un entorno unificado. Su arquitectura desacoplada facilita la evolución tecnológica y la integración de nuevos modelos de lenguaje o servicios bibliográficos, fortaleciendo su potencial como asistente inteligente para la investigación académica.

IV. Resultados Obtenidos del Proyecto

Al término del proyecto, se alcanzaron los principales objetivos planteados, manteniendo la coherencia con la solución inicial y con los criterios establecidos en el análisis SMART.

En primer lugar, se concretó el desarrollo de una plataforma modular asistida por un agente que facilitó la exploración, análisis y síntesis de información académica. Su uso permitió optimizar el tiempo destinado a la revisión de literatura, especialmente en las etapas iniciales de investigación.

El agente inteligente construido funcionó como núcleo de la solución: gestionó de manera adecuada las herramientas especializadas y permitió que el usuario interactuara mediante consultas en un ambiente conversacional. Con ello, se consiguió automatizar tareas clave dentro del flujo de revisión bibliográfica y ofrecer respuestas fundamentadas en los recursos proporcionados.

El sistema demostró su capacidad para procesar documentación y brindar salidas útiles de manera autónoma (por ejemplo, la generación resumida del material proporcionado). La incorporación de elementos interactivos, como la visualización de relaciones entre artículos, enriqueció la experiencia del usuario al ofrecer mayor claridad contextual entre las fuentes que desea utilizar.

Además, la estructura modular permitió integrar cada una de las herramientas sin afectar el funcionamiento general, confirmando la escalabilidad prevista durante el diseño. La solución se mantuvo estable bajo los requerimientos definidos, asegurando una experiencia fluida.

Finalmente, la plataforma se consolidó como un entorno que asistió de manera efectiva en la revisión de literatura

académica y científica, centralizando procesos clave y proporcionando mecanismos claros y confiables de recuperación y síntesis de información. Esto confirmó la correspondencia entre los objetivos inicialmente planteados y los resultados alcanzados, validando la pertinencia de la solución desarrollada.

V. CONCLUSIONES Y TRABAJO A FUTURO

A. Conclusiones

1. El desarrollo del presente proyecto permitió consolidar una arquitectura modular basada en agentes, capaz de integrar de forma ordenada y escalable las herramientas necesarias para apoyar los procesos de investigación académica, demostrando la pertinencia de este enfoque para sistemas orientados a tareas complejas.
2. La implementación de herramientas especializadas posibilitó la consulta, procesamiento y recuperación de información científica mediante flujos definidos y verificables, fortaleciendo la capacidad del sistema para asistir al usuario en actividades que requieren análisis documental estructurado.
3. La incorporación de estrategias de prompt engineering, junto con el diseño de un árbol de decisión para el ruteo de tareas y mecanismos de post-validación, favoreció la generación de respuestas más alineadas con los objetivos de veracidad, completitud y consistencia establecidos, lo que evidenció su valor como elementos metodológicos para la construcción de agentes confiables.
4. La inclusión de un componente iterativo mediante un patrón Singleton de verificación contribuyó a mejorar la calidad del razonamiento interno, al permitir que el agente evaluará sus propios procesos y corrigiera potenciales inconsistencias antes de entregar una respuesta final al usuario.
5. El diseño y desarrollo de interfaces de usuario acorde con las tres fases operativas del agente posibilitó una interacción clara y fluida, donde la lógica del sistema se integró adecuadamente con mecanismos de visualización y control, favoreciendo así el uso intuitivo de sus funcionalidades.
6. El uso de funciones asíncronas y manejadores de eventos resultó determinante para asegurar una ejecución eficiente y responsiva, dado que permitió orquestar paralelamente las solicitudes del usuario y las operaciones internas del agente sin comprometer la estabilidad de la plataforma.
7. La gestión especializada de los mensajes generados por el agente permitió dar soporte a formatos específicos e interacción directa con el código fuente de la interfaz, lo que incrementó la adaptabilidad del sistema frente a distintos tipos de peticiones y escenarios de uso.
8. La combinación de una arquitectura basada en módulos desacoplados, principios de responsabilidad y salvaguardas orientadas a la veracidad permitió incrementar la confiabilidad y pertinencia de los resultados, consolidando al sistema como una base adecuada para el desarrollo futuro de agentes inteligentes especializados.
9. Al concretar la integración entre modelos de lenguaje, herramientas analíticas y componentes de interfaz, la solución resultante demostró ser capaz de articular procesos de recuperación, análisis y síntesis de información científica dentro de un entorno unificado.
10. Desde una perspectiva social, el proyecto contribuye a fortalecer el ecosistema de investigación al brindar a los investigadores una herramienta que facilita el acceso, análisis y aprovechamiento del conocimiento científico, reconociendo y potenciando el valor de los desarrollos generados dentro de la academia. Al integrar tecnología utilizada comúnmente en entornos industriales y ponerla al servicio de la comunidad investigadora, la plataforma promueve una relación más equitativa entre producción científica y aplicación tecnológica, impulsando así la transferencia de conocimiento y la generación de nuevas oportunidades de innovación.

B. Proyección futura

El estado actual del proyecto, con un flujo funcional y herramientas operativas, representa un prototipo robusto en forma de MVP *(Minimum Viable Product)* que valida su concepto central. No obstante, para alcanzar la visión completa de un agente autónomo de investigación, se proponen las siguientes mejoras:

- Transparencia en las respuestas: Indicar el recurso de origen utilizado en las respuestas generadas mediante RAG, fortaleciendo la trazabilidad y la confianza en la información proporcionada.
- Gestión de usuarios: Incorporar un sistema de registro, autenticación y almacenamiento persistente que conserve los recursos, conversaciones y proyectos de cada usuario.
- Infraestructura y escalabilidad: Migrar hacia una base vectorial más robusta —como Milvus o Qdrant— y desplegar la plataforma en la nube para garantizar su funcionamiento con múltiples usuarios de forma simultánea.
- Expansión de herramientas cognitivas: Integrar nuevas tools orientadas a la clasificación temática, identificación automática de referencias y generación de resúmenes comparativos, ampliando las capacidades analíticas del asistente.
- Para garantizar la escalabilidad y el soporte multiusuario en paralelo, se requiere implementar un

canal de comunicación bidireccional mediante WebSockets. Esto permitirá el streaming en tiempo real de las respuestas del LLM y la notificación asíncrona de las tareas de alto consumo (RAG/resúmenes), mejorando drásticamente el rendimiento y la experiencia de usuario bajo carga.

Estas mejoras consolidarán el objetivo original de Wearch AI: convertirse en un asistente inteligente integral que acompañe al investigador en la etapa de la recopilación de información y facilitar la generación de conocimiento.

En conclusión, Wearch AI ha establecido una base sólida para la evolución de las herramientas de investigación asistida por IA. Su arquitectura modular, su enfoque integrador y su visión orientada a la eficiencia posicionan al proyecto como un referente prometedor en la transformación digital del trabajo científico, con el potencial de reducir significativamente las barreras temporales y operativas que enfrentan los investigadores contemporáneos.

Reconocimientos

El desarrollo del proyecto Wearch AI ha sido un esfuerzo colaborativo que se ha beneficiado significativamente de la invaluable orientación y apoyo de nuestros profesores (doctores). A ellos, expresamos nuestro más sincero agradecimiento:

Al Dr. Jorge Galvez

Agradecemos profundamente su contribución esencial en la delimitación de la idea central del proyecto. El Dr. Galvez nos proporcionó feedback crítico y oportuno sobre nuestra concepción inicial, utilizando su vasta experiencia en el proceso de investigación para compartir herramientas y metodologías reales. Su guía fue fundamental para encauzar el proyecto hacia la viabilidad, ayudándonos a comprender la verdadera naturaleza del problema y las necesidades concretas de los investigadores, lo que consolidó la dirección de Wearch AI.

A la Dra. Karla Ávila.

Extendemos nuestro reconocimiento a la Dra. Karla Ávila por su papel pionero en nuestra formación. Le agradecemos por habernos introducido en el fascinante mundo de la inteligencia artificial y por su apoyo constante durante nuestra trayectoria académica. Su impacto fue directo en la génesis del proyecto, ya que, al otorgarle a Rodrigo (integrante del equipo) la oportunidad de realizar su servicio social en apoyo a la investigación, se le permitió conocer de primera mano el entorno de la investigación real. Esta experiencia fue el catalizador que impulsó la redefinición del proyecto, transformando la idea inicial de Search AI a Wearch AI, en honor al edificio W.

Referencias

[1] OpenAI, “A practical guide to building agents,” OpenAI, 2025, [Online]. Available: https://cdn.openai.com/business-guides-and-resources/a-practical-guide-to-building-agents.pdf

[2] IBM Technology, “RAG vs Fine-Tuning vs Prompt Engineering: Optimizing AI Models,” YouTube. Apr. 14, 2025. [Online]. Available: https://www.youtube.com/watch?v=zYGDpG-pTho

[3] IBM Technology, “Generative vs Agentic AI: Shaping the Future of AI Collaboration,” YouTube. Apr. 21, 2025. [Online]. Available: https://www.youtube.com/watch?v=EDb37y_MhRw

[4] OpenAlex, “OpenAlex Technical Documentation,” OpenAlex Technical Documentation, Oct. 12, 2025. https://docs.openalex.org/

[5] LangChain, “Building Effective Agents with LangGraph,” YouTube. Jan. 27, 2025. [Online]. Available: https://www.youtube.com/watch?v=aHCDrAbH_go

[6] Krish Naik, “Introduction to Undertsanding RAG(Retrieval-Augmented Generation),” YouTube. Aug. 31, 2025. [Online]. Available: https://www.youtube.com/watch?v=fZM3oX4xEyg

[7] Greg Kamradt, “The 5 levels of text splitting for retrieval,” YouTube. Jan. 08, 2024. [Online]. Available: https://www.youtube.com/watch?v=8OJC21T2SL4

[8] LangChain, “Documentation by LangChain,” Docs by LangChain, 2025. https://docs.langchain.com/

[9] N. Campos and M. Oshin, Learning LangChain, 1st ed. O’Reilly, 2025.